

What is AutoTrack?

AutoTrack is a web-based inventory + operations system for managing auto parts in small auto shops, repair networks, and parts warehouses. The point of the project is not to build a massive enterprise ERP, but to learn how to build a realistic multi-module system: inventory correctness, workflows, restocking logic, dashboards, analytics, and team integration.

Core capabilities (what the system must do)

1. Inventory management (parts + truth of stock)- Track parts by SKU, compatibility/fitment metadata, location/bin, and quantity changes over time.
2. Request & fulfillment workflows - Users can request parts; the system supports approval/assignment, and tracks fulfillment status (picked, packed, pickup/delivery).
3. Supplier + restocking automation- Maintain supplier records, restock thresholds, low-stock alerts, order history, and basic reorder workflows.
4. Operational dashboard (real-time visibility) - A “single-pane” view showing current stock, low-stock/high-demand alerts, and at-a-glance operational health.
5. Analytics & reporting - Usage trends, turnover rates, demand summaries, and simple forecasting (even a baseline model is fine).

Who does what (5 student modules)

Student 1: Inventory Management

- Parts catalog CRUD (SKU, description, category, compatibility tags, location)
- Quantity adjustments + movement history (receipts, picks, returns, corrections)
- Validation rules (no negative stock unless backorder mode is explicit)
- API endpoints and DB schema for parts + inventory events

Student 2: Request & Fulfillment

- Request creation, approval flow, and fulfillment lifecycle
- Assignment (who fulfills), pickup/delivery status, timestamps
- Integrations to Inventory: reserving stock, decrementing on fulfillment
- API contract for requests + status transitions

Student 3: Supplier & Restocking

- Supplier directory (contacts, lead time, preferred parts)
- Restock thresholds per SKU/location
- Low-stock alerts + reorder suggestions
- Purchase order history (even simplified: “Order placed → received”)

Student 4: Operations Dashboard

- Dashboard UI (live tables/cards: low stock, high demand, pending requests)
- “Exceptions” view (discrepancies, delayed fulfillment, frequent stockouts)
- Real-time-ish updates (polling or websockets—keep it simple)
- Coordinates UI data contracts with Students 1–3

Student 5: Analytics & Reporting

- Reporting endpoints and UI pages (filters by date range, SKU, category)
- Metrics: usage frequency, turnover, stockout count, supplier lead-time stats
- Simple forecasting (moving average / exponential smoothing is fine)
- Export (CSV) and basic charting

Integration requirements (non-negotiable):

- Students 1–3 must agree on the shared data model (SKU identity, locations, stock movement events).
- Students 2 + 1 must agree on reservation/fulfillment rules (when stock is reserved vs. decremented).
- Students 4 + 5 must align on dashboard + analytics endpoints and consistent naming.

Picking the tech stack (tools) for AutoTrack

AutoTrack is a classic “modern web app”: frontend dashboard + backend API + database.
Optional scanning + charts.

Frontend (dashboard UI)

Recommendation (simple + safest): React- Best fit for complex dashboards and component-heavy UIs.

Suggested UI features:

- Search + filters everywhere (SKU, category, location, supplier)
- Role-based navigation (Inventory vs Fulfillment vs Admin)
- Mobile-friendly pages for scanning workflows

Backend (API + business logic):

Recommendation (simple): Express.js (TypeScript)

- You’ll likely have multiple modules and business rules; TypeScript helps keep contracts clean.

Architecture suggestion:

- Interchangeable parts, part #, description, color

Database (system of record)

Recommendation (strong default): MySQL

- Inventory relationships (parts, locations, suppliers, requests, movements) are relational and benefit from joins + constraints.

Optional: Redis only if you truly need caching/queues; not required for capstone success.

Authentication + roles

You don’t need enterprise SSO. Keep it realistic:

- Email/password + JWT auth
- Roles: admin, manager, tech/fulfillment, viewer
- Add audit logging for sensitive actions (stock corrections, deletions)

Barcode / QR scanning (optional but high value)

Pick one approach:

- USB barcode scanner (plug-and-play, acts like keyboard input) for a “shop counter” workflow
- Mobile device camera (webcam/phone camera) for warehouse picking

Library suggestion from the options:

- If you want both 1D+2D: zxing-js
- If QR-only: jsQR
- If 1D-only: quagga2 ^[OBJ]

Visualization

If using React:

- Recharts is straightforward for dashboards

Otherwise:

- Chart.js or Apache ECharts are flexible across frameworks.

Hosting + deployment

Keep it simple and reliable:

- Dockerize backend + database (dev)
- Deploy to one of:
- Render / Railway / Fly.io (very simple)
- Or a cloud VM (GCP/AWS/Azure) if your course prefers cloud exposure
- Use a managed Postgres if possible (less ops risk)

Development + Deployment (team workflow)

- GitHub repo with branches: main, develop, feature/*
- Issue tracker: every feature is a ticket with acceptance criteria
- API contract first: OpenAPI/Swagger (FastAPI gives it automatically; Nest/Express can generate docs)

CI/CD (minimum viable):

- GitHub Actions: lint + test + build on PR
- Optional: auto-deploy develop to a staging environment

Testing strategy:

- Unit tests for business rules (restock thresholds, state transitions)
- Integration tests for core workflows:
- “Create request → approve → fulfill → inventory decremented”
- “Receive shipment → inventory increased → dashboard updates”

Security basics:

- Validate inputs server-side
- Never trust client-side stock updates
- Protect admin endpoints + rate limit logins
- Store secrets in env vars (not in git)

Proposed semester timeline (aligned to CS 490 dates)

This matches the course cadence (proposal → requirements → architecture → sprints → midterm demo → final).

Week 1 (Jan 13): Orientation + system vision

- Read AutoTrack spec, agree on scope boundaries

Output: 1-page vision + module owners

Week 2 (Jan 20): Project selection + proposal

- Finalize scope + success criteria + team roles

Output: proposal + risk list (data consistency, integration)

Week 3 (Jan 27): Requirements (SRS draft)

- User stories + acceptance criteria for each module

Output: SRS draft + workflow diagrams

Week 4 (Feb 3): Architecture & Design

- Decide stack (React + Nest/Express + Postgres, etc.)
- ERD draft (Parts, Locations, InventoryEvents, Requests, Suppliers, Orders)

Output: architecture doc + ERD + API outline

Week 5 (Feb 10): Sprint 1 planning

- Define “walking skeleton”

Output: working app shell + auth skeleton + DB migrations

Week 6 (Feb 17): Sprint 1 review

- Inventory CRUD + basic request creation (no approvals yet)

Output: demo of part creation + stock adjustment + request creation

Week 7 (Feb 24): Sprint 2 planning

- Add workflow rules + dashboard skeleton

Output: request approvals + fulfillment statuses defined

Week 8 (Mar 3): Midterm demo

- Show core workflows end-to-end:
- Add part + stock
- Create request + approve + fulfill
- Stock updates correctly
- Dashboard reflects it

Output: midterm demo + technical review

Week 9 (Mar 10): Refinement & testing

- Fix bugs found in midterm, add tests

Output: stable core + test suite started

Week 10 (Mar 17): Spring break

Week 11 (Mar 24): Sprint 3 development (advanced features)

- Supplier/restocking alerts
- Scanning prototype (USB or camera)
- Analytics v1 (basic charts + CSV export)

Output: feature-complete system (first pass)

Week 12 (Mar 31): Ethics, security, responsible AI

- Access control review, audit logging, data integrity checks

Output: security checklist + “known limitations” section in docs

Week 13 (Apr 7): Final integration & regression testing

- Full integration across all modules
- Load test-ish: seed database, simulate many requests

Output: release candidate build

Week 14 (Apr 14): Final prep

- Documentation finalization, deployment hardening, demo script

Output: final report + polished demo path

Week 15 (Apr 21): Final presentation & demo

- Live demo + Q&A

Output: final delivered system

Week 16 (Apr 28): Reflection & wrap-up

- Individual reflections + lessons learned

Output: reflection + postmortem

Conclusion

AutoTrack is a strong capstone because it forces real-world coordination: shared data, workflow correctness, role-based access, dashboards, and analytics—without the chaos of “massive scale.” A practical default stack is React + TypeScript backend (Nest/Express) + PostgreSQL, with optional scanning and charting layered on once core workflows are stable.

If you want, I can also produce a one-page architecture diagram + ERD draft (Parts / InventoryEvents / Requests / Suppliers / Orders) in the same style as your CloudArena writeup.